

Install the required libraries:

```
pip install tensorflow matplotlib numpy
```

Import Required Libraries

```
import tensorflow as tf
```

```
import numpy as np
```

```
from tensorflow.keras.applications import (  
    ResNet50,  
    VGG16,  
    InceptionV3  
)
```

```
from tensorflow.keras.layers import (  
    Dense,  
    Dropout,  
    GlobalAveragePooling2D  
)
```

```
from tensorflow.keras.models import Model
```

Task 1: Load ResNet50 and Count Trainable Layers

```
from tensorflow.keras.applications import ResNet50
```

```
# Load pretrained ResNet50
```

```
model = ResNet50(  
    weights="imagenet",  
    include_top=True  
)
```

```
# Print Summary
```

```
model.summary()
```

```
# Count trainable layers
```

```
trainable_layers = sum(layer.trainable for layer in model.layers)
```

```
print("Total Layers:", len(model.layers))
```

```
print("Trainable Layers:", trainable_layers)
```

Example Output

Total Layers: 177

Trainable Layers: 177

Explanation

By default, all layers in a pretrained ResNet50 model are trainable.

Task 2: Use VGG16 as a Feature Extractor

Load CIFAR-10

```
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.applications.vgg16 import preprocess_input
from tensorflow.keras.applications import VGG16
```

```
(x_train, y_train), (_, _) = cifar10.load_data()
```

```
# Select first 10 images
images = x_train[:10]
```

```
# Resize to 224×224
images = tf.image.resize(images, (224,224))
```

```
# Convert to NumPy
images = images.numpy()
```

```
# Preprocess
images = preprocess_input(images)
```

Load VGG16

```
vgg = VGG16(
    weights="imagenet",
    include_top=False
)
```

Extract Features

```
features = vgg.predict(images)
```

```
print("Feature Shape:", features.shape)
```

Example Output

Feature Shape:
(10, 7, 7, 512)

Explanation

Each image is converted into a **7×7×512** feature map.

Task 3: Freeze All Layers Except Last 30

```
from tensorflow.keras.applications import InceptionV3
```

```
base_model = InceptionV3(  
  
    weights="imagenet",  
  
    include_top=False,  
  
    input_shape=(299,299,3)  
  
)
```

```
# Freeze all layers  
for layer in base_model.layers:  
    layer.trainable = False
```

```
# Unfreeze last 30 layers  
for layer in base_model.layers[-30:]:  
    layer.trainable = True
```

Add Custom Classifier

```
x = base_model.output  
  
x = GlobalAveragePooling2D()(x)  
  
x = Dense(  
    256,  
    activation="relu"  
) (x)  
  
output = Dense(  
    5,  
    activation="softmax"
```

```
)(x)
```

```
model = Model(  
    inputs=base_model.input,  
    outputs=output  
)
```

Compile

```
model.compile(  
  
    optimizer="adam",  
  
    loss="categorical_crossentropy",  
  
    metrics=["accuracy"]  
)
```

Count Trainable Parameters

```
trainable_params = np.sum(  
    [tf.keras.backend.count_params(w)  
     for w in model.trainable_weights]  
)  
  
print("Trainable Parameters:", trainable_params)
```

Task 4: Fine-Tune ResNet50 on Food Images

Load ResNet50

```
base_model = ResNet50(  
  
    weights="imagenet",  
  
    include_top=False,  
  
    input_shape=(224,224,3)  
)
```

Freeze All Layers

```
for layer in base_model.layers:  
    layer.trainable = False
```

Unfreeze Last 10 Layers

```
for layer in base_model.layers[-10:]:  
    layer.trainable = True
```

Add Classification Head

```
x = GlobalAveragePooling2D()(base_model.output)  
  
x = Dense(  
    128,  
    activation="relu"  
) (x)  
  
output = Dense(  
    2,  
    activation="softmax"  
) (x)  
  
model = Model(  
    inputs=base_model.input,  
    outputs=output  
)
```

Compile

```
model.compile(  
  
    optimizer="adam",  
  
    loss="categorical_crossentropy",  
  
    metrics=["accuracy"]  
)
```

Example Training

```
history = model.fit(

    train_dataset,

    validation_data=validation_dataset,

    epochs=2

)
```

Report Accuracy

```
print(

    "Training Accuracy:",

    history.history["accuracy"][-1]

)
```

Example Output

Epoch 2 Accuracy:

0.93

Note: The actual accuracy depends on the subset of Food-101 (or another food dataset), preprocessing, and random initialization.

Task 5: InceptionV3 for Binary Classification

```
from tensorflow.keras.applications import InceptionV3
from tensorflow.keras.layers import (
    GlobalAveragePooling2D,
    Dense,
    Dropout
)
from tensorflow.keras.models import Model

# Load pretrained model
base_model = InceptionV3(
    weights="imagenet",
```

```

        include_top=False,
        input_shape=(299,299,3)
    )

    # Freeze all layers
    base_model.trainable = False

    # Custom classification head
    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dropout(0.5)(x)
    x = Dense(128, activation="relu")(x)
    output = Dense(1, activation="sigmoid")(x)

    model = Model(
        inputs=base_model.input,
        outputs=output
    )

    # Compile
    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    model.summary()

```

Changes Made

- Added GlobalAveragePooling2D() before the Dense layers because the convolutional output must be converted into a feature vector.
- Used Dropout(0.5) to reduce overfitting.
- Used a single-neuron output with sigmoid activation and binary_crossentropy loss for binary classification.

Summary

Task Implementation	Output
1 Load ResNet50 with ImageNet weights, print the summary, and count trainable layers	By default, all layers are trainable.

Task Implementation	Output
2 Use VGG16 (include_top=False) as a feature extractor on 10 preprocessed CIFAR-10 images	Produces feature maps with shape (10, 7, 7, 512) .
3 Freeze all InceptionV3 layers except the last 30, add a custom 5-class classifier, and count trainable parameters	Fine-tunes only the final part of the network while keeping most pretrained weights fixed.
4 Fine-tune ResNet50 by unfreezing the last 10 layers and training for 2 epochs on a small food-image dataset	Report the final training accuracy (for example, around 0.93 , depending on the dataset and training conditions).
5 Load InceptionV3 , freeze all layers, add a GlobalAveragePooling2D → Dropout → Dense classification head, and compile for binary classification	Produces a deployment-ready transfer-learning model for binary image classification.